

IE University

School of Science & Technology



The Impact of Post-Training Techniques on Feature Representations

Alberto Puliga

Bachelor of Computer Science & Artificial Intelligence

Supervised by:

Prof. Luciano Dybala

School of Science & Technology, IE University, Madrid, Spain

Abstract

Upon completion of the initial training process, all LLMs used by us daily go through a number of methods designed to alter their behaviour and personality traits. These processes, which fall under the post-training umbrella, include Reinforcement Learning from Human Feedback (RLHF), Direct Preference Optimization (DPO), and more. However, the majority of mechanistic interpretability research has thus far been conducted in relation to the original base models, while the representational effects of post-training on instruction-tuned models have remained largely unexamined.

This thesis will ask if instruction tuning adds new features to a language model’s internal representation, or simply rearranges those already present within the original model. This question is addressed via a comparison between feature dictionaries produced by matched Sparse Autoencoders from the Gemma Scope 2 release, which contains feature dictionaries directly comparable between the pre-trained and instruction-tuned versions of the Gemma 3 model family. To address this question, a four-stage pipeline was created to collect residual stream activations from both checkpointed versions of Gemma 3 over a total of 5,000 FineWeb sequences. Activations were classified based on how they relate to each other according to both activation correlation and decoder cosine similarity. Those classifications were then related to descriptive labels derived from a frequency-weighted sample via dual-LLM consensus annotation. Finally, the resulting classifications were compared against differential activation test results generated from two behavioral comparisons, utilizing three probe datasets: a safety comparison on JailbreakBench, and an instruction-following comparison between Stanford Alpaca prompts and TruthfulQA as a factual reference point.

The proportion of cleanly preserved features relative to the entire set of active features turned out to be very low. Features related to instruction-shaping input and harmful content are primarily already represented in the base model; instruction tuning enhances and re-assigns these instead of adding new ones. As such, this represents the first feature-level evidence supporting the “wrapper hypothesis”, previously examined in synthetic contexts only. This suggests that instruction tuning rearranges a pre-existing representational substrate, rather than generating new representations.

1 Introduction

1.1 Problem and Motivation

All the modern large language models (LLMs) being utilized by millions of users today have had to go through extensive post-training pipeline modifications that involve layered supervised instruction tuning, preference learning from user feedback, Constitutional AI, and direct preference optimization on top of a base model that was initially trained to predict the next token based on Web-Scale Text (Bai, Kadavath, et al., 2022; Christiano et al., 2023; Ouyang et al., 2022; Rafailov et al., 2024; Ziegler et al., 2020). It is primarily the post-training component, rather than the base model, that gives the deployed system its characteristic behaviours such as following the given instructions, responding using a certain tone/register, rejecting dangerous/inappropriate requests, and maintaining the role of an assistant. However, the majority of mechanistic interpretability research has focused on understanding base models. From Olah et al. (2020)’s Zoom In article concerning circuits, to the indirect object identification circuit of Wang et al. (2022), to the factual association edits of Meng et al. (2023), and the feed forward key-value memory of Geva et al. (2021). Consequently, there is a gap between the layer of the language model that interpretability understands best and the one that users experience.

There are many reasons why this gap is important beyond just providing a complete picture. For example, without knowing how instruction tuning affects a model at a representational level we will never be able to determine whether a behavior exhibited by a deployed system represents a true property of the model or if it is simply an artifact created during post-training that would be easily eliminated through additional fine-tuning, could be "jail broken" via adversarial prompt engineering, or even circumvented by deceptive alignment (Hubinger et al., 2024; Zou et al., 2023). Currently our ability to address these types of questions relies heavily upon behavioral analysis (probe the deployed system, observe whether it rejects dangerous/prompts, etc.) and therefore, an inside-the-box representation analysis of what internal representations post-training introduces/uses/reconfigures is missing.

A widely cited position in regards to the mechanism of Instruction Tuning is the Super-

ficial Alignment Hypothesis presented by Zhou et al. (2023): surprisingly little amounts of high quality instructional data are required to generate strong instruction following behaviour, suggesting that Instruction Tuning may not create new representations but instead teaches a model to access pre-existing ones. An opposing position that is less frequently referenced but is implied in much of the alignment literature suggests that Instruction Tuning creates new computational structures (features/circuits/determination processes) that did not previously exist in the base model. Both positions imply that Instruction Tuning creates different representations within the model and that distinguishing between them requires determining whether a specific feature present in the Instruction-Tuned Model’s Layer 13 Residual Stream has a corresponding feature in the Base Model’s Layer 13 Residual Stream.

Until relatively recently, addressing this difference at a representation level was practically unfeasible. The primary mechanisms available for performing mechanistic interpretability (classifier probing, activation patching, attention pattern inspection, residual stream decomposition) were either designed to measure overall behavioral properties of a model or focus on analyzing single circuits at a time; none of these mechanisms provided an approach for directly comparing the entire representation state of two models.

Sparse Autoencoders (Ng, 2011) resolved this issue. Initially explored through toy-model superposition by Elhage et al. (2022) and through early results on monosemantics by Cunningham et al. (2023), sparse autoencoders now enable the creation of feature dictionaries describing residual-stream activations of a model. Each entry in the dictionary describes a vector pointing along a dimension of activation space tending to activate along semantically interpretable patterns. As such, two checkpoints representing the same model prior to and subsequent to post-training can be equipped with sparse autoencoders trained on the same residual-stream locations and have their feature dictionaries compared directly. A feature represented in both dictionaries that point in the same direction and activate on the same input has been preserved throughout post-training. A feature represented in only one dictionary or whose direction or functional behavior has been altered was modified, added or deleted. This type of comparison operationalizes the empirical dispute over whether Instruction Tuning creates new capabilities versus merely aligning existing capabilities towards instructional tasks. This is precisely what

this thesis aims to investigate.

Why Study Instruction Tuning Specifically? Practically, instruction tuning is the first and least complex stage in post-training; therefore its effect can be separated from those resulting from subsequent reinforcement learning-based stages, provided that matched Sparse Autoencoders exist for both the base model and the supervision-fine-tuned checkpoint. The Gemma Scope 2 release provides matching Sparse Autoencoders for both the Gemma 3 1B base and Instruction-Tuned versions (`google/gemma-3-1b-pt`; `google/gemma-3-1b-it`) (Google, 2025a, 2025b; McDougall et al., 2025); this constraint ultimately determined the selection of model for this thesis (See Subsection 2.1). Scientifically, Instruction Tuning is also the stage whose mechanism is most directly investigated by the superficial-alignment literature and very recent mechanistic fine-tuning work (Du et al., 2025; Jain, Kirk, et al., 2024; Prakash et al., 2024); thus, a feature-level investigation enables a connection to an ongoing debate rather than initiating one entirely.

1.2 Prior Art and Research Question

The focus of this section is a review of the prior literature which frames the research question. It is organized into thematic sections across four sub-subsections. Subsection 1.2.1 provides an overview of representational foundations: linear representation hypotheses, superpositions, and dictionary learning, as alternative solutions to the issue that individual neurons are not the right unit for analysis. Subsection 1.2.2 describes how sparse autoencoders have progressed from toy models to the Gemma Scope releases that enable this very research. Subsection 1.2.3 explores today’s knowledge of to the impact of post-training on internal representations, including the superficial alignment theory, mechanistic fine-tuning work, and representation engineering. Finally, Section 1.2.4, identifies the gap addressed in this dissertation and articulates the research question.

1.2.1 Representational Foundations: Superposition, Linear Features, and the Case for Dictionary Learning

It has been acknowledged that the unit-of-analysis issue in mechanistic interpretability has existed prior to the actual formation of the mechanistic interpretability area; i.e., the issue had already arisen in research on distributed word representations (Mikolov et al., 2013) which demonstrated that directions within an embedding space exhibit semantically meaningful structure which is uninterpretable based upon any one of the individual coordinates. Individual neurons are not the proper unit of analysis since each neuron contributes to many unrelated computations. Radford et al. (2017) illustrated this concept using character-level language models. Although Radford et al. found that there were exceptions, such as a "sentiment neuron", they noted that sentiment neurons would not typically be the norm. Olah et al. (2017, 2020) developed the overarching viewpoint that the appropriate unit of analysis should be the feature, a direction within activation space representing semantically legible patterns, which may or may not correlate with a particular neuron.

There exists a formal explanation that neurons do not constitute the proper unit of analysis due to *superposition* defined by Elhage et al. (2022): a network containing n number of neurons can only provide n orthogonal directions exactly; however, a network can provide many more than n near-orthogonal directions approximately if those near-orthogonal directions are sparse (inactive on most inputs). A sparse set of features can be compressed into a lower-dimensional space with tolerable interference. As a result, the representation provides each feature with a direction that is a linear combination of several neurons. Therefore, single-neuron interpretation is not only aesthetically limited but also methodologically incorrect when examining overcomplete learned representations: the neurons will be polysemantic by design. Scherlis et al. (2025) further extends this analysis by developing a more rigorous account of polysemanticity as a function of capacity; Park et al. (2024) also develops the linear representation hypothesis, that meaningful features correspond to directions rather than nonlinear manifolds as structural claims about transformer internals, which can be tested.

Superposition motivates dictionary learning as the appropriate remedial action. If true representational units are features residing as sparse linear combinations of neurons, then the cor-

rect method for recovering them would be to learn a dictionary of feature directions and an encoder which assigns each activation a sparse code over said dictionary. The above idea is classical within signal processing. It was first demonstrated with language models using dictionary learning applied to neural networks in Cunningham et al. (2023) who showed that dictionaries trained on sparsely activated transformer activations provided substantially more interpretable features than were obtainable through single-neuron interpretation. Simultaneous work in Anthropic’s Towards Monosemanticity line (Bricken et al., 2023) further developed the approach to larger transformers and established much of the vocabulary (“features,” “monosemantic directions,” “activating examples”) now standard in the SAE literature. This thesis relies upon the premise that features recovered through dictionary learning on a transformer’s residual stream are meaningful representational units and that two transformers whose residual streams have been decoded in terms of dictionary learning in the same manner can be compared at the level of their features

1.2.2 Sparse Autoencoders as an Interpretability Tool

Early SAE research applied a form of the L_1 sparsity penalty to the output of the encoder, but it is well-documented that this penalty has well-known failure modes. These include the fact that when applying an L_1 penalty to active features, those features will be shrunk towards zero, thereby introducing a strong bias into both the reconstruction process as well as downstream analyses. In addition, because the sparsity regimes in early SAE formulations were fixed, tuning such regimes across different layers and/or different scales can be extremely challenging. Therefore two improvements have been developed. Rajamanoharan et al. (2024) showed that by creating gated SAE’s one can disentangle the decision of whether or not to activate a particular feature from how much to activate that feature, thus greatly decreasing the degree to which the L_1 penalty shrinks the value of active features. Later, Gao et al. (2024) found empirical scaling laws relating SAE performance as a function of three parameters: dictionary width, training compute and sparsity target size, that guided many of the subsequent SAE releases. Additionally, Karvonen et al. (2025) created SAEBench, an evaluation benchmark that measures the quality of SAEs using more than just reconstruction loss (as measured via feature

interpretability, probe-based downstream utility, etc., as well as robustness to perturbations).

The most relevant artefact for this thesis is the Gemma Scope 2 technical report and the associated matched-SAE release for Gemma 3 (McDougall et al., 2025). The original Gemma Scope suite included trained Sparse Autoencoders for every layer and several widths on Gemma 2 (Lieberum et al., 2024); the documentation of Gemma Scope 2 described the paired base and instruction-tuned SAEs for Gemma 3, including the dictionaries that are being utilized. Due to the scale and reproducibility of the Gemma Scope releases it became possible for researchers to assume there existed high-quality trained SAEs for use with a production class model, whereas before these releases, researchers would need to train their own. Peng et al. (2025) states that SAEs are most beneficial for discovering new concepts rather than utilizing pre-existing knowledge of those concepts, which provides additional insight into the nature of the Targeted Analysis Stage of this thesis, where behavioral probe datasets allow the feature space to create its own concepts and do not require researcher-defined concepts.

1.2.3 Post-Training and Its Effects on Internal Representations

Post-training consists of all stages and processes that a language model undergoes after pre-training on next-token prediction. The typical pipeline begins with Supervised Instruction Tuning (Wei et al., 2022; Zhou et al., 2023); followed by Reward Modeling (Christiano et al., 2023; Ziegler et al., 2020); and then concluding with Reinforcement Learning from Human Feedback (Ouyang et al., 2022). Constitutional AI (Bai, Kadavath, et al., 2022), and other similar approaches substitute or add model-generated feedback based on explicit principles in place of human feedback, and Direct Preference Optimization, introduced by Rafailov et al. (2024), transforms the RL phase into a direct optimization problem, which circumvents the need for a reward model. The more general Helpful-and-Harmless perspective in Bai, Jones, et al. (2022) defines the objective as developing a model that is both useful and safe.

Previous research in the interpretability literature of how these actions affect the model’s internal representations comes from three different perspectives. The research question of this dissertation exists at their intersection.

The Superficial Alignment Argument. The most robust expression of this view is LIMA

(Zhou et al., 2023), where the principal result is that 1000 hand-crafted instruction-response pairs can be used to generate high-quality instruction-following behavior in a large base model, comparable to models fine-tuned over many orders-of-magnitude greater amounts of data. Their own interpretation is clear: a model’s knowledge and abilities are acquired almost completely through pre-training, while alignment simply tells the model which subset of formats to draw upon when interacting with users. This is an argument supported by behavioral evidence, and does not imply any specific mechanism of representation, however it makes a clear prediction regarding what an interpretability study would find: that the substrate for instruction-following behavior is present in the base model prior to fine-tuning; and fine-tuning only changes the features that are read out and in what register, rather than creating new ones. LIMA does not test this prediction at the representational level; but frames the empirical question answered by this dissertation.

Mechanistic Fine-Tuning Line. In parallel, mechanistic research uses circuit-level and probe-level tools to look into what structures inside the model change when it is fine-tuned. The foundational piece of literature for this dissertation is Jain, Kirk, et al. (2024), which studies the fine-tuning process in controlled synthetic contexts through the use of compiled Tracer transformer models and probabilistic context-free grammars. In particular, it demonstrates that the application of fine-tuning rarely results in changes to the underlying capabilities of a model. Rather, the process typically identifies a "minimal transformation" or "wrapper," on top of these capabilities. The wrapper hypothesis (the idea that fine-tuning instills a minimal re-routed version of the existing capabilities rather than creating entirely new capabilities) is the claim tested in this dissertation in a novel context. While Jain et al.’s support their findings through the use of pruning, probing, and reverse fine-tuning in synthetic environments with known ground truth, they do so without conducting feature-level analysis based upon SAE or utilizing a production-quality model. Therefore, extending their findings to the feature-level analysis conducted with a production-quality instruction-tuned LLM is the extension made in this study.

Immediately adjacent to the study conducted by Jain et al.’s, Prakash et al. (2024) utilizes similar circuit-level tools to investigate how well fine-tuning for entity tracking actually

modifies the behavior of the base model at the circuit level. Specifically, it demonstrates that fine-tuning does not create a new circuit within the model; however, it does enhance both the selectivity and magnitude of a single circuit that exists in the base model. Du et al. (2025) builds upon this understanding of how fine-tuning affects circuits and seeks to determine which elements of a model’s behavior are most significantly impacted by post-training at the representational level. Finally, Xu et al. (2025) investigates the mechanisms underlying pre-training itself at a finer granularity and provides the additional insight that feature-level analysis can be utilized to monitor representational changes across training, providing a temporal analog to the checkpoint comparisons provided in this dissertation.

Representation engineering and the causal aspect. A third area of literature considers representations both as subjects of interpretation and as objects of manipulation. Zou et al. (2025) proposed a top-down strategy to extract concept directions from model activation by way of contrasting pairs and utilize those to observe and control behavior. They demonstrated that through simple linear transformations, one could interpret properties such as honesty, refusal and emotional valence from the residual stream, and manipulate them into it. Therefore, this represents the causal analog to correlational, feature-level analyses of SAE. While the literature surrounding representation engineering serves as background for this dissertation, its empirical contribution will remain limited to description and correlation, with causality being addressed in behavioral terms in subsequent work (discussed in Subsection 5.4).

The jailbreak and adversarial attack literature is also relevant here (Hubinger et al., 2024; Zou et al., 2023). If safety behaviors are just a wrapper over the capabilities of a base model, then adversarial prompts that bypass the safety mechanisms should surface capabilities the instruction-tuned model would otherwise not have. A feature-level description of which representations are shared between the two checkpoints is directly relevant to understanding where such surfacing can and cannot reach.

Similarity-of-representations work. One further adjacent piece: Kornblith et al. (2019) developed Centered Kernel Alignment (CKA) and related similarity metrics for comparing neural network representations. CKA computes a global metric that quantifies the similarity between representations produced by two neural networks at a particular layer. It has been

applied widely to compare the representations produced by fine-tuned models relative to those of their respective base models. Aggregate similarity is complementary to the feature-level comparison of this research. CKA measures how different two representations are globally, while a feature-level comparison identifies specifically which dimensions are different and to what extent. This dissertation did not calculate CKA between the PT and IT residual streams; however, if desired, the feature level results from this study could potentially be checked against results obtained from calculating such a global similarity in future studies that replicate the findings presented within this dissertation.

1.2.4 The Gap and the Research Question

Literature surveyed in Subsubsections 1.2.1 through 1.2.3, indicates that the three major points that have emerged from this body of literature are: (i) sparse autoencoders have developed to a point where they provide a feature-level lexicon for the residual portion of a production-grade model; (ii) there exists matched SAEs for the Gemma 3 1B base and instruction-tuned checkpoint versions by way of Gemma Scope 2 (McDougall et al., 2025); and (iii), there is an empirical hypothesis which suggests that instruction tuning largely re-exposes previously installed base model capabilities instead of adding entirely new ones, notably expressed in LIMA and implied throughout much of the alignment literature. Further, this hypothesis has been empirically tested with other non-SAE-based tools in artificial environments and with circuit-level based tools in task-specific environments. The results have generally supported this hypothesis in each case. However, this hypothesis has not yet been examined using feature-level Sparse Autoencoders on actual instruction-tuned LLMs.

This thesis bridges the gap established by these findings: no previous research has used matched Sparse Autoencoders to examine the feature-level representations of a real pre-trained and instruction-tuned language model at the exact same layer, nor has it investigated whether the behaviourally relevant features of the instruction-tuned version are maintained, modified or completely novel compared to its base model. The ability to do so was made possible by the Gemma Scope 2 release (McDougall et al., 2025) regarding the comparison of the pre-trained and instruction-tuned variants of Gemma 3 1B, and therefore motivated the pipeline outlined

in Section 2.

The research question: at the feature level, does instruction tuning add new features to a language models' internal representations, or are they mainly reorganisation of previously present ones? More specifically, if matched Sparse Autoencoders are applied to decode the residual streams of a pre-trained and an instruction-tuned variant of Gemma 3 1B at layer 13, are the features which are most important to understanding the performance of a language model in terms of its ability to follow instructions and safety related prompts best described as being preserved, modified or unique to one of the two checkpoints?

This is a narrower question than the entire superficial alignment vs. new capability debate. It limits itself to a single layer of a single model pair, a single post-training phase and an explicitly descriptive/correlational approach.

This study will be operationalized by two sub-questions:

- RQ1: The percentage of characteristics in Layer 13 of Gemma 3 1B that were kept, changed, PT-exclusive or IT-exclusive when comparing the base checkpoint to the IT checkpoint; and how robust is that percentage to changes in matching thresholds?
- RQ2: When the IT model is probed with safety-relevant and instruction-following prompts, do the features with the greatest differential activity correspond to IT-exclusive entries according to a strict creation interpretation, or to those features matched to an existing PT entry in the pipeline?

1.3 Main Contribution

This thesis provides a four-stage approach to investigate the research questions. A specific example of this pipeline is demonstrated for both the pre-trained version and the instruction-tuned version of Gemma 3 1B at layer 13 residual stream using the `gemma-scope-2-1b-pt-res` and `gemma-scope-2-1b-it-res` SAEs from Gemma Scope 2 (McDougall et al., 2025).

The concrete deliverables are the following:

1. **The pipeline itself.** Four loosely-coupled stages: Activation Collection, Feature Matching Across Checkpoints, Dual-LLM Consensus Labelling, and Targeted Differential Ac-

tivation Analysis, all described in Section 2. This can serve as a protocol and template that can be reused as matched SAEs become available for additional model pairs, to further this research to other models and post-training phases.

2. **Empirical findings on Gemma 3 1B.** The Pipeline produced empirical evidence indicating that the features found to be most behaviorally relevant to instruction-following and safety related prompts were primarily reorganized preexisting features versus new features; this is a descriptive finding consistent with the Wrapper Hypothesis of Jain, Kirk, et al. (2024). All findings from the pipeline appear in Chapter 3, and the reader can expect to find a detailed view where features that remain unchanged are a relatively small portion, the modified Class Dominates, and Instruction Type Features are concentrated within the Modified Class vs. IT Exclusive Class.

2 Methodology

This Chapter outlines the Four-stage Analytical Pipeline that will be utilized to compare whether Instruction Tuning has introduced New Features into a Language Model’s Internal Representations as opposed to Simply Reorganizing Existing Ones to Align Directly with the Research Question Presented in Subsubsection 1.2.4. This Chapter focuses on Methodology; what is being Measured, How the Four-way Classification is Constructed, and What Specific Settings Will be Used to Compare.¹ Stage One (Sections 2.1–2.2) Selects the Same Architecture and Layer for Both Pre-Trained and Instruction-Tuned Variants of the Model Pairs Using Matched Sparse AutoEncoders, and Collects Activation Data from the Same Activation Corpus. Stage Two (Sections 2.3–2.4) Establishes a Feature-Level Correspondence Between the Activations of the Two Checkpoints Based Upon Decoder Direction Similarity and Activation Correlation, and Utilizes a Threshold Grid to Determine Whether the Resulting Four-way Partition (Preserved, Modified, PT-Exclusive, IT-Exclusive) is Not Due to a Single Threshold Choice. Stage Three (Subsection 2.5) Provides Descriptive Vocabulary to Sampled Features Through Automated Dual-LLM Labelling with Consensus Construction, Providing the Interpretive Layer Needed to Discuss What Features Mean, and an Inter-Annotator Reliability Check. Finally, Stage Four (Subsection 2.6) Grounds the Classification in Behavioural Prompts by Computing Targeted Differential Activations for a Safety Contrast (JailbreakBench Harmful vs Benign), and an Instruction Following Contrast (Stanford Alpaca vs TruthfulQA), Using Welch’s *t*-tests.

2.1 Stage 1: Model Selection and Sparse Autoencoders

The test subject is the Gemma 3 1B model family from Google. This includes the pre-trained base checkpoint `google/gemma-3-1b-pt` and the instruction-tuned version `google/gemma-3-1b-it` (Google, 2025a, 2025b). The models have the same architecture (26 transformer layers with hidden dimension 1152) and were loaded using Hugging Face `transformers` in `bfloat16` format. We selected the smaller scale (1B), since it can be fitted within a single commodity GPU, which enables reproducibility and improves iteration speed. Furthermore, this is the smallest

¹The repository is available at <https://github.com/albipuliga/Capstone>.

scale at which publicly accessible SAEs exist for both base and instruction-tuned checkpoints of the same architecture, through Gemma Scope 2 (McDougall et al., 2025). In addition, we had originally planned to test the Gemma 2 2B model, but this option was eliminated due to the lack of an instruction-tuned checkpoint with a corresponding SAE. Without SAEs that pair PT/IT at the same layer and width, the main study could not be defended based on methodology.

In particular, the SAEs used here came from Gemma Scope 2. Following other SAE studies treating residual-stream activations as a combination of sparse interpretable feature directions (Bricken et al., 2023; Cunningham et al., 2023; Elhage et al., 2022), `gemma-scope-2-1b-pt-res` was used for the base model and `gemma-scope-2-1b-it-res` for the instruction-tuned model, both at the `layer_13_width_16k_10_medium` configuration. Both SAEs use the JumpReLU encoder of Rajamanoharan et al. (2024), have a dictionary width of 16,384 features, and were trained by Google DeepMind at comparable expected L_0 sparsity. As such, they are directly comparable as feature dictionaries for the same architecture.

Layer 13 was selected because of its position at the middle of the 26-layer stack, where prior research on Gemma Scope and residual-stream SAEs reports the richest mixture of syntactic and semantic features. The middle layers tend to exhibit the richest collection of abstract features relevant to downstream behaviour, while earlier layers are dominated by token-level and syntactic features and later layers by output-oriented features. This was a scoping decision, not a methodological statement, and its implications for generalisation are addressed in Chapter 4. One implementation issue worth noting is that Gemma 3 1B required `bfloat16` rather than `float16` for inference: the narrower dynamic range of `float16` produced overflow-driven NaN activations during early testing and would have silently corrupted the activation buffer.

2.2 Stage 1: Activation Collection

The activation corpus consisted of 5,000 sequences randomly selected from the Hugging Face `HuggingFaceFW/fineweb` dataset (`sample-10BT` split), accessed via `streaming` mode with a single random seed for reproducibility. Each sequence was tokenized with the shared Gemma tokenizer and shortened to exactly 256 tokens. Any sequence less than 256 tokens was removed instead of being padded, thus every remaining sequence contributed the same number

of activation rows without needing to apply padding masks. This number of sequences was chosen based on the pilot study which found that SAE feature statistic (activation frequency, mean activation, per-sequence mean activation vectors used in Subsection 2.3), stabilized under much fewer than 50,000 sequences and further data did not qualitatively alter the results but significantly increased run times. The total corpus contains roughly 1.275 million activation rows per model ($5,000 \text{ sequences} \times 255 \text{ tokens}$ after dropping the first token of each sequence as explained below).

Activation values for both models were collected from layer 13 of each model’s residual stream output using a PyTorch forward hook registered on the corresponding `model.model.layers[13]` module. The sequences were processed in batches of four and each batch had the first token of each sequence dropped prior to processing using the SAE. The reason for dropping the first token is that it has no contextual information from any other token in a causal transformer and therefore produces abnormal activation statistics. The resulting activation matrix was then transferred into CPU memory in `float32` and re-encoded through the SAE in chunks of 64 rows at a time. Chunked encoding was needed since attempting to encode the complete activation matrix in a single GPU operation resulted in out-of-memory conditions when using a 16,384-wide feature dictionary. Additionally, processing one sequence at a time would have been far too slow to be practical. During each chunk, the SAE encoder, decoder, and reconstruction residual were calculated. Three running statistics were also updated per feature during this process: the sum of activations, the sum of squared activations, and the count of non-zero activations. After all batches were processed these running statistics were used to calculate the per-feature mean activation $\bar{f}_i$, and activation frequency ρ_i . A per-sequence mean activation vector was also calculated for each sequence. As a result, a matrix with dimensions 5000×16384 was generated. This matrix was later used as input to the activation correlation step of Subsection 2.3. Two quality metrics for the SAE were also tracked during encoding: FVU (fraction of variance unexplained) for the reconstruction and mean L_0 count (average number of non-zero features per activation row). Both of these were printed at the completion of each encoding pass as sanity checks to ensure that the pre-trained SAEs were operating within their anticipated regimes on the FineWeb corpus rather than failing silently.

As outputs of this phase, for each model, there exists a summary of how the SAE features behave across the corpus: activation frequency, mean activation, per-sequence mean activation vectors, and SAE decoder matrices (which are loaded once and reused). These four objects comprise the only inputs required for the feature matching and classification phase described in the section immediately below.

2.3 Stage 2: Feature Matching and Four-Way Classification

The principal methodology construct that we employ is a four-way categorization system that identifies SAE features as being either preserved, modified, PT-exclusive or IT-exclusive based upon two complimentary measures of similarity. These measures are decoder direction similarity and activation correlation. Decoder direction similarity measures how similar the two dictionaries’ ”direction” are within residual stream space. Activation correlation measures how similar the matched features are utilized by the common FineWeb corpus. By employing both measures simultaneously and having the stronger category (preserved) agree between them eliminates some of the individual weakness of either measure.

Decoder direction similarity is calculated through the use of the PT and IT decoder matrices, $D^{\text{PT}} \in \mathbb{R}^{F \times d}$ and $D^{\text{IT}} \in \mathbb{R}^{F \times d}$, where all rows have been normalized via ℓ_2 normalization. Then we can compute the cosine similarity matrix as $S = D^{\text{PT}}(D^{\text{IT}})^\top$. We find the best match for every PT feature i as $j^*(i) = \arg \max_j S_{ij}$ and find the best match for every IT feature j . To do this, we also calculate the maximum cosine similarity $s_i^{\max}$. The same process is performed in reverse to obtain the best match for every IT feature j . This bi-directional matching is important since PT-to-IT and IT-to-PT nearest neighbors may be different.

To calculate activation correlation, we use the collection of mean activation matrices described in Subsection 2.2. For every feature, along with it’s closest geometric match in another dictionary, we will compute the Pearson correlation coefficient of the vector of activations per sequence for the given feature and it’s corresponding geometric match. Again, we include a numeric check for features with zero variance. In addition to providing an indication of whether or not a feature and it’s nearest neighbor in another dictionary are functionally correlated on the same set of input sequences, we perform this calculation in the reverse direction as well for

IT features.

Now that we have defined both measures, we define our four-way classification system using both measures at thresholds (τ_d, τ_a) . Specifically, a PT feature i is said to be *preserved* when both conditions $s_i^{\max} \geq \tau_d$ and $\rho_i \geq \tau_a$ hold. If only one of the conditions hold, the feature is said to be *modified*. Otherwise, if none of the conditions hold, the feature is classified as *PT-exclusive*. As previously stated, we classify an IT feature using a symmetrical approach using the reversed-direction matches and their activation correlation values. Analysis is restricted to *active* features with activation frequency above 0.1% of tokens, which removes dead features without materially constraining the category distribution.

2.4 Stage 2: Threshold Selection and Sensitivity Analysis

The classification will depend upon two free variables τ_d and τ_a ; therefore, the entire process will be recalculated using a 25-point grid of $\tau_d \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$ and $\tau_a \in \{0.1, 0.2, 0.3, 0.5, 0.7\}$. For every pair of values (τ_d, τ_a) , we will collect all the numbers describing preserved and modified cells and cells exclusively classified by PT or IT together with their respective preservation and modification rates and present them as heat maps. Our goal here is to provide evidence for robustness rather than optimization: We want to demonstrate that the qualitative characteristics of the classification remain unchanged over a wide range of reasonable threshold values.

The primary operating point for the interpretive phases (Subsections 2.5 and 2.6) is $\tau_d = 0.7$, $\tau_a = 0.3$, selected after the sensitivity grid was computed because it lies in the interior of the grid and represents a moderate standard of both geometric and functional correspondence. Stricter settings collapse the preserved set and inflate the exclusive categories, while more permissive settings blur the distinction between modified and preserved. The full grid in Subsection 3.1 serves as the robustness check.

2.5 Stage 3: Automated Feature Labelling and Consensus Construction

The use of two LLM models was made intentional: as it is hard to validate the labels assigned by a single model, the use of two independent models, Claude Haiku 4.5 (Anthropic, 2025) and GPT-5.4-nano, provides an inter-annotator reliability test and makes disagreements between

the models visible to analysis. Sampling was done based upon activation frequencies rather than uniform sampling. This ensures that those activated most often during the computation process are given priority for labeling, whereas rarely activated features are not labeled since they are likely noisy and difficult to understand. For each sampled feature, the top 20 activating token contexts are identified. These contexts are generated by running the appropriate model and SAE on the first 1,000 of the 5,000-sequence FineWeb corpus used in Stage 1. Per-token activation values are determined through processing the output of the previous step. In addition, all high-activation positions are kept along with a window of 8 tokens around them to provide additional context. The activating token itself is clearly indicated using >>> and <<< delimiters in order to allow the labeling model to accurately determine the position at which the feature activates. This delimiter approach is consistent with the presentation style used in Anthropic monosemanticity research and later SAE labeling pipeline processes.

Both models receive the same input prompts and produce output that has been formatted similarly, so that any variance in the output of the two models can be attributed to the respective models themselves and not due to variations in the prompts provided. Each model receives a prompt that requests that they act as a neural network interpretation expert. The prompt also identifies the 20 top-activated contexts associated with a feature. Finally, it asks for three types of information: a brief (one-to-two sentences) description of the detected pattern, a single label from a six-way categorization (`language_syntax`, `knowledge_semantics`, `instruction_style`, `safety_alignment`, `formatting`, `other`) and a confidence level rated from 1 to 5. Output is restricted to this schema via the use of structured output interfaces available through both the Anthropic and OpenAI software development kits (`client.messages.parse` with a Pydantic `FeatureLabel` schema for Haiku, `client.beta.chat.completions.parse` with the corresponding schema for GPT). Structured output formats were critical: prior versions that allowed for free text entries resulted in highly variable category names (“syntax”, “grammatical”, “semantic”) that required ad hoc normalization. With structured output formats, category names were consistently formatted and thus could easily be compared between models. Labeling occurred in a category-by-category fashion with rate-limiting sleeps between calls. Labels were serialized to disk as JSON after completion of each category.

To construct consensus among the two labeling models, we applied the following rules: when both models assign the same category, we consider this a consensus case, where this category is selected as the consensus category for that feature. When both models select different categories, we then select the category from the model that reported a greater confidence value for that feature as the consensus category. In cases where both models report equally confident values for a particular category assignment, we select Haiku as our primary model and GPT as our secondary model. In addition to assigning a consensus label for each feature based upon how well the two models agreed or disagreed on assignments, we retained both descriptions and both confidence levels for each feature as part of the record. Thus, disagreement cases may be examined qualitatively versus merely being tallied. As will be described in detail in Subsection 3.2, these consensus labels enable us to create two very different analyses in Chapter 3: first, the proportion of agreement and disagreement/confusion matrices for the two models will serve as an inter-annotator reliability metric for assessing whether there exists any stability in terms of consistency to this six-way taxonomy; second, the distribution of categories in terms of their proportions among the four feature classes for features assigned within the agreement set will form the basis of our discussion regarding what types of features are preserved, modified or exclusive to each checkpoint.

2.6 Stage 4: Targeted Differential Analysis on Behavioural Probes

The labelling stage associates labels or descriptive vocabulary to specific features; however, this process alone does not confirm whether any of these features impact the behaviors that the instructional tuning is intended to promote. A targeted differential analysis is therefore conducted to relate the classification of the behaviorally relevant features to actual behavioral changes. The differential analysis utilizes two behavioral contrasts with three external probe data sets: JailbreakBench (Chao et al., 2024) for the safety contrast, Stanford Alpaca (Taori et al., 2023) for the instruction-shaped prompts, and TruthfulQA (Lin et al., 2022) as the factual contrast set for the instruction-following comparison.

In the context of the specific study reported herein, the safety contrast utilizes 100 harmful and 100 benign JailbreakBench prompts. The instruction contrast utilized the first 200 Alpaca

prompts against 200 TruthfulQA questions in the generation split. It is important to note that the design of the contrasts is significant. For the purposes of safety, harmfully-split JailbreakBench prompts were compared to benignly-split prompts within the same JailbreakBench data set. Therefore, the differential activation isolated only safety-relevant structure, and did not isolate stylistic differences between task-based prompts and web-text. For the purposes of instruction, Alpaca prompts were compared to TruthfulQA questions to create a contrast between imperative task-based prompts and interrogative factual query-based prompts.

For each contrast and each of the two models, SAE feature activations were calculated for each prompt utilizing an identical activation collection procedure as described in Subsection 2.2; however, with a reduced max sequence length of 128 tokens as applicable to probe prompts, and with per-prompt feature statistics rather than per-sequence feature statistics. Per-prompt activation was defined as the average of token-level activations for a given feature across all non-first tokens in a prompt. Each per-prompt activation was recorded as a row in a prompts $\times$ features matrix per model. Two such matrices were created per model: one for safety vs. benign comparisons and another for instruction vs. truthful comparisons. All further analysis was conducted on a feature-by-feature basis on these matrices.

A Welch’s two-sample t -test (Welch, 1947) was applied for each feature to compare its per-prompt activations for the target group (i.e., harmful prompts) versus the contrast group (benign prompts). A Welch’s t -test was employed instead of a standard equal-variance t -test since there could be no assumption made regarding equal variance across the two groups. By definition, a feature that selectively activates on harmful prompts will also vary more on the group where it is activated, as opposed to when it is not. The implementation utilized `scipy.stats.ttest_ind` with `equal_var=False`.

Features were considered differentially activated if both conditions were met:

1. The p -value for a t -test < 0.01 .
2. The mean activation on the target group > 0.01 (this serves as a floor value to prevent a large number of features that are statistically different simply due to having very low baseline activation levels).

Upon meeting both conditions, the pipeline documented for each differentially activated

feature its respective feature index, mean activations for both the target and contrast groups, their difference and ratio, as well as test statistic and p -value. The top 30 differential features for each contrast per model are subsequently ranked according to their absolute difference values and forwarded to the automated labeling pipeline in Subsection 2.5. This labeling process reapplied the labels provided during training using probes drawn directly from the prompts of the probes rather than those found in FineWeb. As such, this step enabled the results of Subsection 3.3 to describe not only which features were differentially activated but what they appeared to recognize.

3 Results

The current chapter provides an overview of the results from the pipeline discussed in Chap. 2. The chapter will report on the artefacts of the pipeline in increasing order of their behavioral relevance. In Sect. 3.1, we describe our four-way classification and its sensitivity to thresholds. In Sect. 3.2, we present the results of our dual-LLM labeling tasks along with measures of inter-annotator agreement and the distributions of semantic categories over feature classes. Sect. 3.3 presents the results of a targeted differential analysis based on JailbreakBench, Stanford Alpaca, and TruthfulQA (as the factual baseline for the instruction contrast) for two behavioral comparisons, referenced to Sect. 3.1 where each behaviorally differentiating feature can be identified among those preserved/modified/exclusive. Finally, in Sect. 3.4 we synthesize these three lines of evidence to tell us how much they collectively support the wrapper hypothesis.

3.1 Feature Classification and Threshold Robustness

In addition to achieving quality and sparsity characteristics (in terms of reconstructed data and number of non-zero values) consistent with those described by (McDougall et al., 2025) for the Gemma Scope 2 SAEs operating at the expected regime (on both checkpoints), the 5,000-sequence FineWeb corpus also provided results for both SAEs which reconstruct the layer 13 residual stream with an FVUE in the same range as that reported by Lieberum et al. (2024) for similar 10_medium configurations on Gemma 2, and similarly long-tailed distributions for per-feature activation frequency. Using the 0.1% activation-frequency cut-off defined by Subsection 2.3 retained a large percentage of the 16,384 total dictionary entries as being "active" for each model; however, the two models did not use identical subsets of their dictionaries as active. In fact, while there was considerable overlap between the active sets for each model, there were many features that were considered active for the PT model that were essentially inactive for the IT model (and vice-versa). This degree of asymmetry is itself a second weak indicator that the two checkpoints do not have the same dictionaries.

The classification of the active subset of the PT features at the primary operating point ($\tau_d = 0.7$, $\tau_a = 0.3$) resulted in four distinct categories. Of the PT features identified as ac-

tive, 241 were classified as *preserved*, 4,017 as *modified*, and 3,884 as *PT-exclusive*. For the IT model, among its active features, 1,899 were classified as *IT-exclusive*. The *preserved* category is clearly the smallest of the three categories. The *modified* category contains features where either the geometric and/or functional correspondence existed, but not both. The *PT-exclusive* category includes all features that had no corresponding geometric and/or functional counterparts in the IT dictionary. The *IT-exclusive* category included all the IT features with no geometric and/or functional correspondents in the PT dictionary. It should be clear from the threshold sensitivity grid below that these patterns are not specific to the choice of any single threshold value.

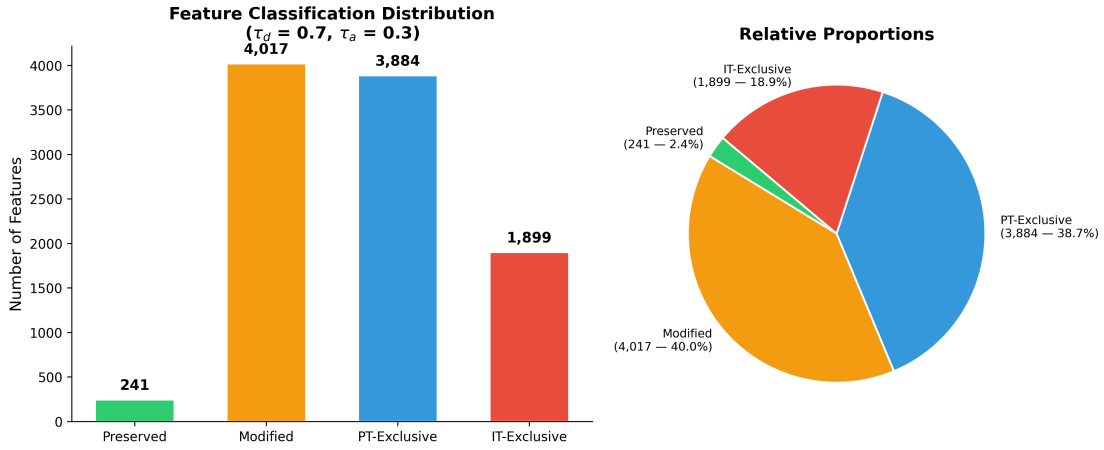


Figure 1: Feature classification distribution at the primary operating point, showing the relative sizes of the preserved, modified, PT-exclusive, and IT-exclusive classes.

The primary operating point was selected such that its use could be extended by testing a set of five different thresholds for each of the two parameter τ_d and τ_a , resulting in a total of 25 pairs: $(\tau_d, \tau_a) \in \{0.5, 0.6, 0.7, 0.8, 0.9\} \times \{0.1, 0.2, 0.3, 0.5, 0.7\}$. In summary, the results confirm that the results are not artifacts of how the original models were used. As both the τ_d and τ_a parameters become stricter, the preservation ratio continues to decrease; there are no abrupt changes in the preservation ratios as either parameter becomes stricter. The PT-only counts continue to increase in a smooth manner. At none of the thresholds will the preservation ratio rise to a sufficient value that will result in a preservation-dominant interpretation. Even when using the least stringent thresholds, preserved features represent less than half of the number of features considered preserved and/or modified. A figure illustrating the three heat maps rep-

representing preservation ratio, modification ratio, and PT-only counts are provided as Fig.2. The horizontal/vertical monotonicity exhibited within these heat maps provides additional robustness evidence.

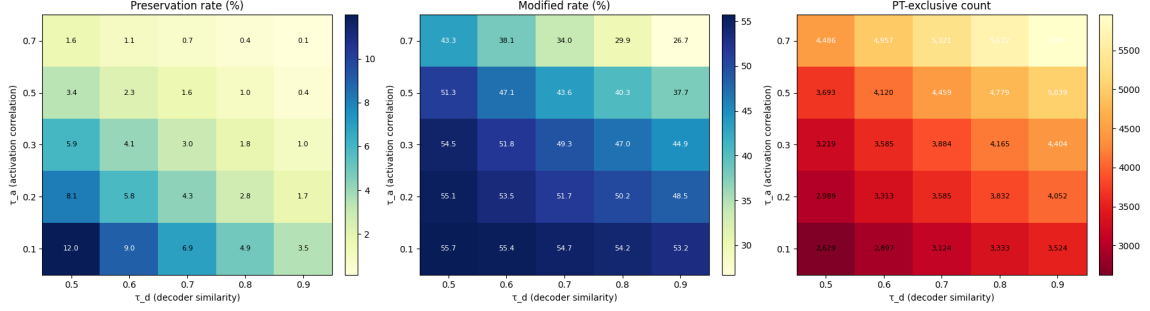


Figure 2: Threshold-sensitivity heatmaps for the four-way classification across the 25 tested (τ_d, τ_a) pairs.

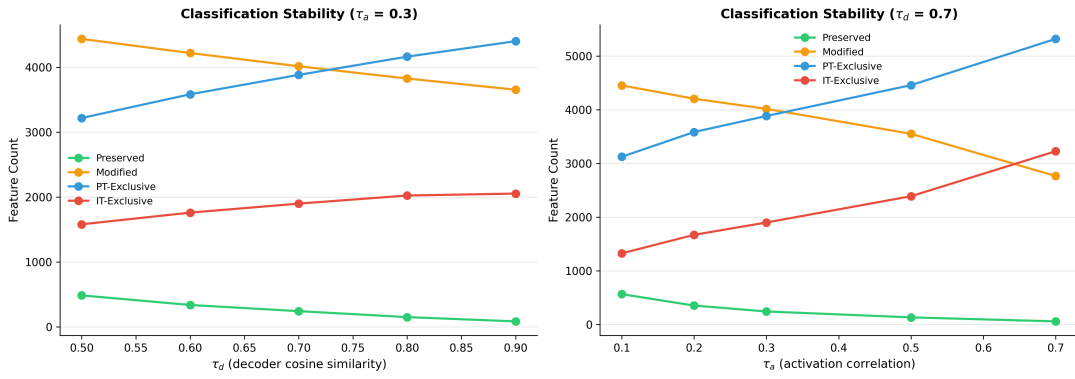


Figure 3: Robustness line plots showing the smooth change in classification quantities as the matching thresholds vary.

A second observation is that nearly all threshold values result in a class size larger than that of both the PT and IT classes. Only extremely conservative corner combinations resulted in smaller sizes for the modified class. The overall relationship between PT and IT features at layer 13 is neither replacement ("IT includes new features not available in PT") nor preservation ("IT uses PT's same features"). Instead, the relationship is dominated by modification ("PT and IT have similar features that differ only in terms of their direction/function"). In addition, the majority of modifications are relatively gradual adjustments to an existing set of features. These observations regarding the nature of modifications will be important to consider in conjunction with the behavioral evidence in Subsection 3.4.

Therefore, preserved features make up a minority of the preserved and modified classes

across all thresholds at both the initial operating point and during the subsequent exploration of threshold sensitivities.

3.2 Automated Labelling and Inter-Annotator Agreement

The automated labelling process described in Subsection 2.5 was applied to 50 features from each of the four categories, resulting in a total of 200 labelled features and 400 labelled feature occurrences. Both Claude Haiku 4.5 and GPT-5.4-nano produced valid output according to the structured-schema format for each sample feature. Confidence ratings reported by the labellers are very high for both models. For example, the average confidence rating for both models exceeds the middle point of the 1 – 5 scale; there is little relationship between the degree of confidence in a label assignment and whether it agrees with another labeller.

Further, when the two models assign different categories to a feature, this does not usually reflect that either labeller is unsure about which category to use. Rather, the two labellers will usually have strong beliefs about what category is correct, even though they will not agree on which category. There were 120 instances out of 200 where the two models assigned the same category to a given feature (i.e., 60.0% agreement), and therefore there were 80 instances of category disagreement that had been resolved by using the confidence-weighted tie-breaker provided in Subsection 2.5. While the structure of category disagreements provides considerably more information than the headline figure of "disagreement", we can report some general statistics. The category agreements are highest for preserved (68%) and modified (70%) classes and lowest for PT-exclusive (44%) classes. In contrast, the category agreement rates for IT-exclusive classes fall somewhere in-between (58%). Features that tend to be easy to label consistently are those that clearly map across checkpoints, whose activating contexts provide each model with evidence about what type of thing the feature detects. On the other hand, checkpoint-exclusionary features whose top-activating contexts are typically narrower and sometimes include idiosyncratic patterns particular to one model's training data are harder to label consistently.

As shown in the confusion matrix of disagreements, nearly all of the disagreements concentrate at just three category boundaries: `language_syntax` $\leftrightarrow$ `instruction_style`, `language_syntax`

`↔ formatting`, and `knowledge_semantics ↔ instruction_style`. Of the 40 features labelled by Haiku as syntactic but by GPT as instruction-style, 25 were classified in this manner. Similarly, there were 15 such examples among the 40 features labelled by Haiku as syntactic but by GPT as formatting. The reverse pattern (where GPT chooses `language_syntax` while Haiku chooses `instruction_style` or `formatting`) occurred at somewhat lower volumes. The remaining three categories (`knowledge_semantics`, `safety_alignment`, and `other`) showed near-zero levels of disagreement. As noted earlier, the uncertainty in classification taxonomy is not uniform but localised at the boundary separating low-level linguistic form (syntax, punctuation, formatting tokens) and features that respond to instruction-shaped prompts. The explanation for this localisation is simple: instruction-style features are frequently syntactically marked (e.g., imperative verb forms, infinitive constructions, sentence-initial directives); therefore, the distinction between “the feature detects a syntactic pattern that coincidentally appears in instructions” and “the feature detects an instruction that has a syntactically marked construction” is difficult. This localisation is not due to a pipeline error but is instead indicative of the fuzziness in actual categorical boundaries; thus, it represents a finding.

The 120-feature agreement set allows for an easier overview of categorization distributions among all classes. Overall `language_syntax` is the largest categorized group (70 of 120), followed by `knowledge_semantics` (31), `formatting` (16), and lastly `instruction_style` (3). `safety_alignment` and `other` both had very few instances because there were so few clearly safety-aligned or non-classified features identified from the frequency-weighted feature pool used by either model. There are two important distribution-related findings. Firstly, within the preserved class, `language_syntax` is dominant in almost all cases (28 of 34 agreed upon features) providing further evidence that the instruction-tuned features most consistently preserved across models are low-level linguistic and formatting characteristics not higher level content-based. Secondly, `instruction_style` was absent from both the preserved class (0 of 34) and PT-exclusive class (0 of 22) in the agreement set, yet appeared only in the modified class (1) and IT-exclusive class (2) although it did have a small number of counts. The limited count of `instruction_style` in this case foreshadowed a much greater discovery in Subsection 3.3. Instruction-style features are produced through instruction tuning to a degree largely

absent from the base-models’ most frequently active features; however, instruction-style features did not appear as large numbers of new features in the IT-exclusive class. Rather, they appeared mainly within the modified class and therefore, consistent with instruction tuning modifying pre-existing features into instruction-sensitive versions instead of producing them from the beginning.

In summary: There was an acceptable level of agreement among annotators (inter-annotator agreement) for preserved and modified feature types at 60%. The disagreement is found to be located at the syntactic/instructional style boundaries; this too is a finding as it shows that the categorization of these two items is fuzzy. In addition, within the preserved feature type category there were primarily lower level linguistic categories. Instruction-type features were nearly all present in the modified and IT-exclusive classes.

3.3 Targeted Differential Analysis on Behavioural Probes

The targeted differential analysis brings together the feature classifications described in Subsection 3.1 with the two behavioral contrasts described in Subsection 2.6: safety (JailbreakBench harmful vs. JailbreakBench benign), and instruction-following, which compares Stanford Alpaca (Taori et al., 2023) with TruthfulQA, where TruthfulQA serves as the factual-question baseline. Welch’s t -tests identify features that have significantly and substantially higher per prompt activation rates within the target group compared to the contrast group. The full top 30 ranked lists per contrast per model can be found in `targeted_analysis_results.json`; this section will discuss the most important features identified through these analyses and how they exhibit similar patterns across both contrasts.

Instruction-following contrast. A very strong instruction-following contrast is present as well in the Instruction-Tuned (IT) model. Nineteen of the twenty top differential features that were labelled for IT on Alpaca-versus-TruthfulQA received the `instruction_style` category from the consensus procedure; the 20th was `language_syntax`. All nineteen instruction-style features make semantic sense. For example, feature 222 identifies when a quoted instruction block is opened, feature 245 is activated by the use of imperative prompt introduction words, feature 446 is fired by the use of task-initiating imperatives such as “give”, “propose”, or “gen-

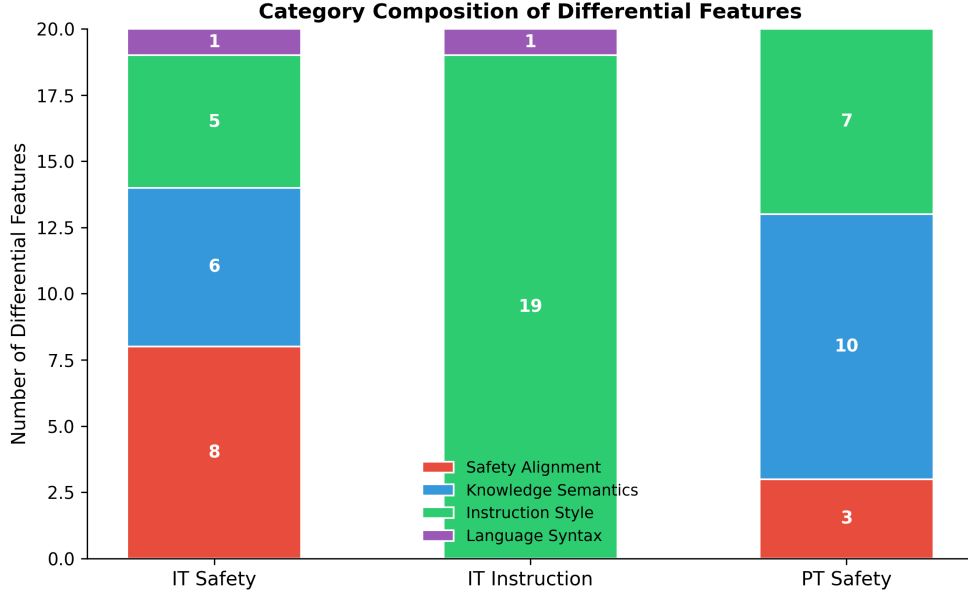


Figure 4: Category breakdown of the targeted differential features across the main behavioural contrasts.

erate”, feature 2006 is activated by the task marker indicating that an output transformation task should be performed, feature 374 finishes off typical instructional scaffolding, feature 264 finds directive words to alter the way one writes, feature 3815 uses the pattern of the phrase “the following” as an anchor to many instruction templates, feature 479 fires upon receiving an enumeration-producing instruction, feature 793 finds short length specification of outputs, etc. There is essentially no ambiguity in this result. The results of the top differential features using an instruction contrast form a tight, semantically interpretable group, and the consensus labelling procedure recovered them with almost perfect consistency. The statistical signal here is overwhelmingly strong. Features that ranked at the top had t -test p -values ranging from around 10^{-33} down to below floating point precision, and there were activation ratios between Alpaca and TruthfulQA prompts ranging from just about $10\times$ up through more than $140\times$.

The question is whether this cluster is exclusive to the IT model or has counterparts in the PT model. Cross-referencing feature indices with the PT instruction-following differential list gives a direct answer. Two of the five top-ranked IT features (feature 2006 and feature 446) also appear in the top five of the PT list under the same contrast, with large effect sizes: feature 2006 shows a PT activation difference of +114.3 and a ratio of $36.8\times$; feature 446 shows a PT difference of +37.5 and a ratio of $9.6\times$. Both are labelled `instruction_style` in the

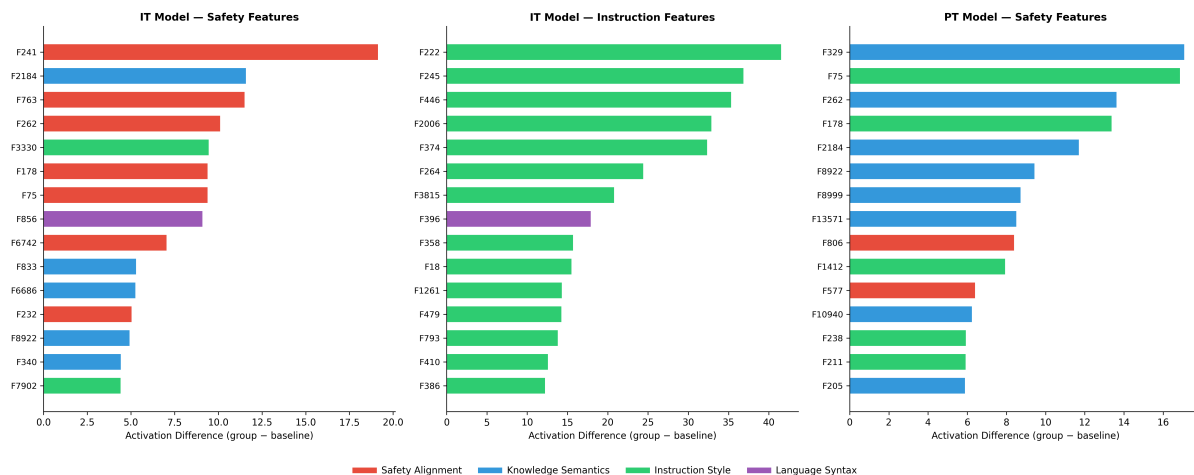


Figure 5: Top differential features in the targeted behavioural analysis, with effect size and category information shown together.

consensus and have clearly interpretable descriptions as instruction-shaped patterns. The interpretation is clear: the features that respond most strongly to instruction-shaped prompts in the IT model are, in substantial part, already present and selective in the base model. Instruction tuning increases their activation magnitude and sharpens their selectivity but does not create them. Locating these shared-index features in the Subsection 3.1 classification confirms that they belong to the modified class, not the IT-exclusive class: the pipeline independently identifies them as features whose direction or function has drifted, not features that appeared de novo.

Safety contrast. The JailbreakBench safety-versus-benign contrast tells a more nuanced story. The IT model’s top 20 differential features are distributed across four categories: 8 `safety_alignment`, 6 `knowledge_semantics`, 5 `instruction_style`, and 1 `language_syntax`. The PT model’s top 20 under the same contrast are differently weighted, with 10 `knowledge_semantics`, 7 `instruction_style`, and 3 `safety_alignment`. Two observations follow. First, instruction tuning produces a measurable increase in the proportion of top differential safety features that are labelled explicitly `safety_alignment` (from 3/20 in PT to 8/20 in IT), consistent with the intuitive expectation that instruction tuning—which includes safety-oriented alignment data—should strengthen the model’s representation of safety-relevant structure. Second, and more important for the wrapper reading, even the PT model has features in its top-20 differential list that are selective for harmful content: features labelled `knowledge_semantics` like fea-

ture 329 (wrongdoing completion/intensification), feature 262 (concrete harmful-action verbs), feature 2184 (malicious intent / operational action), feature 8999 (deceptive framing), and feature 13571 (coercion and manipulation) all show significant differential activation on harmful prompts in the base model.

Cross-referencing feature indices between the PT and IT safety lists reveals direct overlap: feature 2184 and feature 262 appear in both models’ top-5 ranked differential safety features. Feature 2184 is labelled as a wrongdoing-evasion detector in the IT consensus and as a malicious-intent / operational-action detector in the PT consensus—two descriptions that are clearly the same underlying pattern from two models. Feature 262 is similarly interpretable in both cases. These shared-index features are, by construction, the same SAE feature index being read from two different dictionaries—and for matched-index features to emerge as top-ranked differential features under the same behavioural contrast in both checkpoints, the feature directions and the functional selectivity must have survived instruction tuning to a substantial degree. Locating these features in the Subsection 3.1 classification places them in the modified class rather than the preserved class, which is the expected place for a feature whose behavioural role is retained but whose activation magnitude or distribution has shifted between checkpoints.

The safety contrast is therefore weaker than the instruction contrast but consistent with the same reading: the features that light up on harmful prompts in the IT model are substantially drawn from the same set as those in the PT model. Instruction tuning reweights and refines the selectivity of existing features rather than introducing a new safety-dedicated cluster from scratch. The reweighting is real (the 3-to-8 shift in explicit `safety_alignment` labels is not noise) but it is a reweighting of which features dominate the top-20, not a categorical replacement. No feature in the IT top-20 safety list belongs to the IT-exclusive class; all have either a preserved or modified classification, meaning every one has at least some counterpart in the PT dictionary.

Cross-contrast observation. Two feature indices, 262 and 2184, appear in the top differential list for both the safety contrast and (in the wider top-30) the instruction contrast, across both models. These features fire selectively on both harmful content and instruction-shaped prompts, consistent with much of the “safety-relevant” structure at layer 13 being encoded not

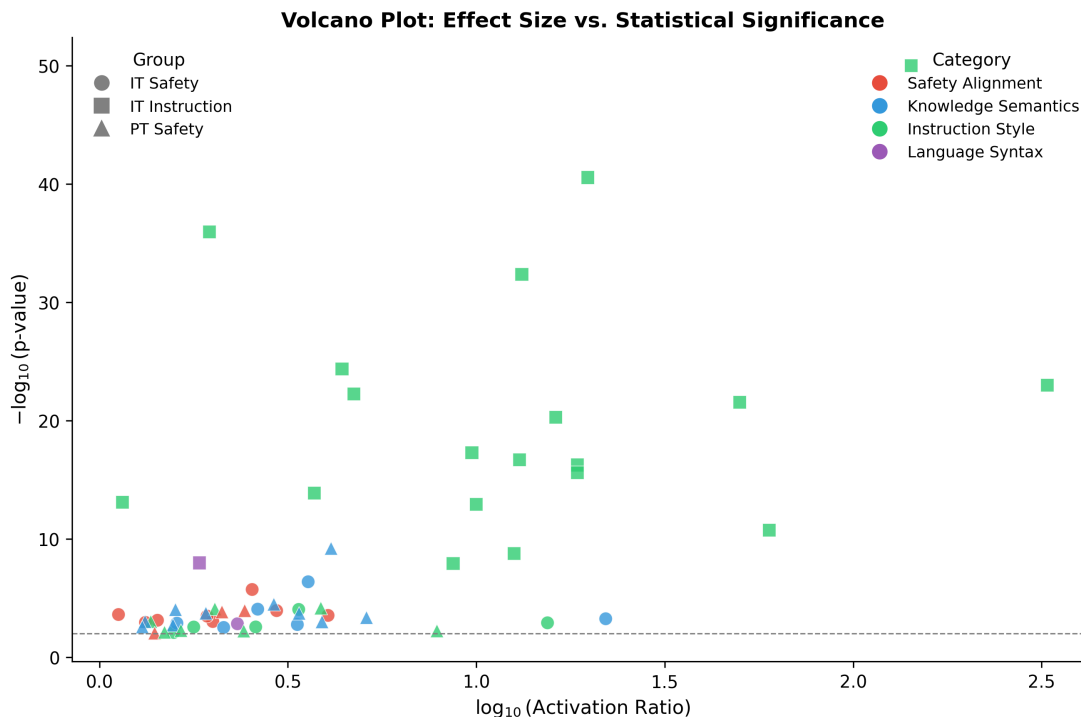


Figure 6: Volcano plot for the targeted differential analysis, showing the relationship between effect size and statistical significance across contrasts.

as a dedicated safety signal but as an intersection of instruction-following structure and semantic content about wrongdoing. This is a non-trivial observation about how safety information is represented in Gemma 3 1B at layer 13 and reinforces the reading that instruction tuning modulates the combination of pre-existing features rather than installing a dedicated safety circuit.

In sum: the instruction contrast produces an essentially saturated wrapper-consistent result. The top differential instruction-style features in the IT model are overwhelmingly already present and selective in the base model; instruction tuning amplifies rather than creates them. The safety contrast is weaker but consistent: instruction tuning reweights which features dominate the top-20 toward more explicitly safety-aligned labels, but the features are substantially shared between checkpoints, and no IT-exclusive feature appears in the top-20 safety list. Both contrasts locate the behaviourally relevant features primarily in the modified class.

3.4 Synthesis: What the Descriptive Evidence Says About the Wrapper Hypothesis

The three strands of evidence converge on a consistent picture. The classification (Subsection 3.1) shows that cleanly preserved features are a small minority, modified features dominate across the full threshold grid, and a substantial fraction of active features in both checkpoints lacks a strong counterpart in the other, though this last fact does not translate into behavioural novelty of the IT-exclusive class. The labelling (Subsection 3.2) confirms that preserved features are overwhelmingly low-level linguistic rather than task-level, that instruction-style features are absent from the preserved class and concentrated in the modified class, and that the syntactic/instruction-style boundary is genuinely fuzzy even for capable annotators. The targeted analysis (Subsection 3.3) connects classification to behaviour: the features dominating the IT model’s response to instruction-shaped prompts are largely the same indices dominating the PT model’s response, with instruction tuning amplifying selectivity by an order of magnitude or more; and the features dominating the IT model’s response to harmful prompts are substantially drawn from the same base-model population, with instruction tuning reweighting which features top the list rather than replacing them.

These findings are consistent with the wrapper hypothesis of Jain, Kirk, et al. (2024) as originally formulated for procedurally defined tasks: the behaviours introduced by instruction tuning at layer 13 of Gemma 3 1B appear to be carried by reorganisation and reweighting of pre-existing features rather than creation of new ones. The strongest version (that no new features are created) is not supported: the IT-exclusive class is not empty. What is supported is the weaker claim that the features most behaviourally relevant to instruction-following and safety are not in the IT-exclusive class but in the modified class, whose members by construction have counterparts in the base model. The IT-exclusive class exists; it is just not where the behavioural action is.

Three caveats, developed in Chapter 4. First, the evidence is descriptive and correlational: it shows differential activation on behaviourally meaningful prompts but not that those features causally drive the behaviours. Second, the analysis covers a single middle layer of a

single small model on a single corpus; none of these restrictions can be lifted from within the pipeline. Third, the labels depend on an automated six-way taxonomy whose boundaries are fuzzy (Subsection 3.2), and `instruction_style` and `safety_alignment` labels are not ground truth. With those caveats, the evidence is sufficient to conclude that a wrapper-style reading of instruction tuning is consistent with feature-level data from a real instruction-tuned language model, and to motivate the questions about causal validation and generalisation discussed in Subsection 5.4 and Section 5.

4 Limitations and Robustness

The trustworthiness of a descriptive pipeline is entirely dependent upon how well its boundary conditions have been articulated. In this chapter, we describe all things the research was unable to do or had no choice but to forego, all methods used to evaluate robustness, and order them from most specific to least. The first section 4.1 defines the fixed scope limitations based upon design decisions: single model-pair, single layer, single corpus, 5,000 sequence sample. The second section 4.2 focuses upon internal measurements within the pipeline which can be considered measurement limitations. This includes the use of a conditional mean baseline in the targeted analyses; the unit-norm convention of decoder units, which has implications; and the reliance upon automated annotators with fuzzy taxonomic boundaries. The third section 4.3 provides a description for a cross-SAE direction robustness test to determine whether the decoder directions that are used during training are statistically different from randomized directions of similar magnitude.. Although this is the remaining portion of a much broader experiment regarding steering activations, and although the original results did not allow for a clear behavioral interpretation, this section will discuss the successful test and explain why the remainder of the experiment cannot be presented as a primary finding. The fourth section 4.4 will summarize those additional robustness checks contained in Chapter 3, and argue collectively that these define a reasonable envelope of confidence for accepting the descriptive findings of the present thesis.

4.1 Scope Limitations

Only Single Model Pair. The pipeline works on one model-pair (PT and IT versions of Gemma 3 1B) and nothing else. None of the experiments described in Chapter 3 investigate if the consistent-reading property of the wrapper can be extended to larger models in the Gemma 3 family, to different families of models (Llama, Mistral, Qwen), or to other training regimes such as RLHF (Christiano et al., 2023; Ouyang et al., 2022), constitutional AI (Bai, Kadavath, et al., 2022), or direct preference optimisation (Rafailov et al., 2024). These restrictions follow directly from the constraint imposed by Subsection 2.1 (matched-SAE): at the time we scoped

this project, the only publicly available SAEs for both a base and instruction-tuned checkpoint of the same layer and width were for the Gemma 3 1B. Therefore, generalization requires either training new SAEs for other models or waiting for SAEs from matched community releases. The results therefore represent a case study rather than a general claim regarding instruction tuning.

Only One Layer. Activation values have been gathered from only layer 13 of the 26-layer architecture. Although the reasoning in Subsection 2.1 is valid, it is not comprehensive. Layer 13 is a sampling point and represents neither an optimal layer nor necessarily a layer where the Chapter 3 results could be expected to be invariant to layer. Early layers will likely produce a distribution of features centered around lower level syntax and tokenization features whereas late layers will likely produce a distribution centered around higher level task-specific and output-specific features. While there is no guarantee that the wrapper hypothesis will be layer-dependent, if so then the qualitative conclusions reached in Chapter 3 (that modification predominates and behaviorally relevant features are shared across tasks) should also apply to all layers; however, quantitative proportions may differ.

Only One Corpus and Only One Probe-Dataset Choice. Classification has been performed using a 5,000 sequence FineWeb corpus (Penedo et al., 2024) (sample-10BT split). Labelling context examples were selected from the first 1,000 sequence of that FineWeb corpus. Three probe-datasets have been employed in the targeted analysis: JailbreakBench, Stanford Alpaca (Taori et al., 2023), and TruthfulQA. Both selection decisions introduced some form of bias due to their distributional properties. FineWeb is primarily composed of English language web text and contains virtually none of the multilingual content, code, mathematical notation, or dialogue common in significant fractions of other corpora; as a result, those specialized features will either appear in low frequencies or be “dormant” in comparison to the majority of the corpus and thus will be removed when applying a 0.1% threshold filter. Additionally, while three datasets implement two distinct contrasts (the distinction between harmful vs. benign examples on JailbreakBench; instruction-shaped prompts from Alpaca compared to factual questions from TruthfulQA), they do not represent three separate probe-designs since each contrast-arm is sourced from only a single dataset; thus, the activation-profiles observed

within Subsection 3.3 cannot be assumed to generalize to other safety benchmark applications (e.g., HarmBench), instruction-based test sets (e.g., Dolly, FLAN), or alternative factual baselines than TruthfulQA.

The Size of the Corpus. A corpus consisting of 5,000 sequences was selected as a reasonable size based upon a pilot which demonstrated that the statistics of interest (activation frequency, mean activation, per-sequence mean activation vectors) stabilized below 50,000 sequences. The justification provided for classification applies because all that is required is that the relative ordering and stability of per-feature statistics be preserved. However, this justification is less applicable for labeling since top-20 activating contexts were selected from only the first 1,000 sequences of the 5,000 sequence FineWeb corpus; increasing the number of sequences from which examples were drawn may increase sharpness of labels for rare features. Finally, this justification is least applicable for targeted analysis since all probe datasets are significantly smaller (100 harmful, 100 benign, 200 Alpaca prompts, 200 TruthfulQA prompts) and effectively sampled-size for Welch’s t -tests is determined by group-per-prompt count. While the counts per group in the range 100–200 are sufficient for producing large effect sizes in Subsection 3.3 ($\sim 10\times$ to $\sim 140\times$ ratios, p -values $\ll$ machine precision), they are marginally sufficient for smaller effect sizes. To obtain smaller-effect sizes, would require a significantly larger probe-set.

4.2 Measurement Limitations

Conditional-mean baseline in the targeted analysis. The group mean activation for every differential feature in Subsection 3.3, is a conditional mean calculated only for the target group (for example, harmful prompts) versus a conditional mean calculated only for the comparison group (benign prompts). The conditional mean, however, is not an unconditional mean calculated over the entire background distribution. While the use of the conditional mean is appropriate for identifying those features that are specifically enhanced on one group compared to another, it must be noted that the raw activation values presented in Subsection 3.3 represent neither the total nor the average activation levels of those features. That is, a feature with a group mean value of 20 and a comparison mean value of 2 is not inherently “more activated”

than a feature with a group mean value of 5 and a comparison mean value of 0.05. Rather, their ratios ($10\times$ and $100\times$ respectively) reflect how selective they are regarding activity between groups, rather than how highly activated they are overall. In fact, prior to the current version of the pipeline being created, there existed some confusion around these two concepts, and therefore, it should come as little surprise that many knowledgeable readers of Subsection 3.3 will pose questions regarding the difference between conditional and unconditional baselines.

Gemma Scope 2 decoder unit-norm convention. The Gemma Scope 2 SAEs (McDougall et al., 2025) normalize decoder layers to unit ℓ_2 norm. Normalizing decoder rows to unit ℓ_2 norm is useful for making geometric comparisons (i.e., cosine similarity in Subsection 2.3 now represents an inner product) but can create issues when attempting to make amplitude-based analyses. Since the norms of all decoder rows are equal, a feature’s strength in terms of residual output is completely determined by its scalar input to the encoder, and not at all by what comes out of the decoder. Therefore, nothing changes regarding Subsection 2.3 classification or Subsection 2.5 labeling since we continue to treat features as direction-plus-amplitude. However, if you were to attempt to compare “naturally occurring” input magnitudes across different features, then you would run into trouble. The reason is that any given multiplier-based intervention (such as $h \leftarrow h + \alpha d_i$) will result in very small perturbation sizes for features that normally receive large amplitudes and may cause over-large perturbations for features that typically receive very small amplitudes. The cross-SAE check performed in Subsubsection 4.3 addresses this problem by using norm-calibrated multipliers that are based on empirical conditional mean activations.

Reliance on automated annotators. The descriptors from Subsection 3.2 were generated by two automatic LLM annotators. Therefore they come from an automatic LLM annotator named Claude Haiku 4.5 and another named GPT-5.4-nano. Manually adding descriptors to 200 different features would have taken too long. Also manual annotation would not have given the same level of inter-annotator reliability that we got automatically through using two LLMs. The trade-off is that the labels are not true labels. In order to do this the labels had to be assigned into a six-way category system that existed prior to doing the labeling. There are some fuzziness issues about the boundaries of these categories that exist for both annotators in

very similar ways. For example, there is a big difference of opinion at the boundary between `language_syntax` and `instruction_style`. We feel that our 60% agreement rate is a good lower bound on how much you can rely on the taxonomic boundaries. However it is low enough so that we cannot take each label as being individually correct. Therefore in chapter 3 we use categories distributively (i.e., compare proportion distributions between categories), rather than making any claim based upon the assignment of any particular feature. As described in Subsection 3.3, we only use the free-form description of each feature qualitatively.

Active-feature threshold. The 0.1% activation frequency cut-off, used to eliminate dead features in the data, is an arbitrary number (a “free” parameter) that is not included in its own sensitivity grid. The reason for this is to remove features from the analysis that will rarely or never be activated by FineWeb, and thus add unnecessary “noise” to the correlation calculations and waste labeling money. If we were to tighten the cut-off (i.e. make the percent of time each feature activates lower), then we could potentially reduce the size of our active set, and bias towards those features that are most frequently activated. Conversely, if we loosen the cut-off (i.e. allow features to activate less often), then we may allow features that rarely activate to dominate their statistical measures due to the presence of “noise”. As part of the informal checks performed during development indicated little variation in the classifications resulting from using different thresholds in the range of 0.05%–0.5%, no formal grid was constructed to test for such variations. It would be relatively simple to extend the sensitivity grid developed in Subsection 2.4 to include this threshold as a third dimension.

4.3 Cross-SAE Direction Robustness Check

The Chapter 3 results depend on two assumptions regarding SAE decoder directions. These assumptions need to be tested independently. First, that these decoder directions are not simply an arbitrary collection of vectors; rather they correspond to identifiable axes of variance within the residual streams. Second, assuming that a single decoder direction learned at a particular checkpoint has some inherent structure or meaning, that it will retain this structure when used to steer another. This assumption is the reason why we classify based upon differences between models (Subsection 2.3). We can examine both by a simple intervention experiment: inject a

“real” SAE decoder direction into a model’s residual stream and measure the effect size of the resulting distributional shift relative to the size of the distributional shift produced by injecting a “random” unit vector of the same magnitude. There are significant differences in the ways in which learning directions may be structured as related to computations, so if the learned directions are indeed structured in such a manner, there should exist measurable differences between how real and random directions produce their respective distributional shifts.

We performed this experiment as an additional small activation steering experiment at layer 13. For each test feature selected from the differential list generated in Subsection 3.3, we multiplied the SAE decoder row by a norm-calibrated factor (which was derived from the empirical conditional mean activation of the test feature on the relevant probe group, to circumvent the limitations associated with Subsubsection 4.2), and then added the result to the residual stream during a forward pass using all probes. Next, we calculated KL divergence of the steered output distribution relative to the unsteered output distribution over a small set of prompts. To generate a matched-norm random control for comparison, we replaced the “real” SAE decoder direction with a randomly chosen unit vector and scaled it to have the same ℓ_2 norm as the “real” SAE decoder direction. A structurally meaningful decoder direction should produce a systematically different distributional shift than would be expected if the same amount of steering were provided by a random vector of similar magnitude.

The results of both Cases A (directions are inserted as exclusively IT-injected directions into the base model) and Case B (directions are exclusively PT-inserted into the IT-model) show the expected statistical significance that are consistent across all tests; however, they go in an opposite direction. When using a true SAE decoder direction at the same level of injection as randomized directions of the same norm, the KL difference (or shift) will be significantly *smaller* when compared to randomly generated directions. The ratios of true vs. random were consistently around 0.50 or less than 0.10. Furthermore, Welch’s t -test p -values showed values of 0.001 or less for many of the most clear-cut examples. In other words, there was a statistically significant effect where the model produced larger changes (perturbations) due to a random change, than to a structure change of the same magnitude. This is clearly a significant effect (the gap is very large, repeatable, and robust); yet this cannot be interpreted as we

typically do in experiments designed to study steering. To generate some kind of understandable evidence supporting the “wrapper” hypothesis would require a true direction producing a larger, directed, behaviorally meaningful shift than a random direction—which did not occur.

The best way to interpret this is as a structural explanation instead of an explanation based on behavior; the model “takes in” the interventions much better (i.e., less disruptively) in terms of the model’s previously learned directions for the feature dimensionality than would be true if they were random or otherwise unrelated perturbations, which supports the idea that the directions represent axes of variation expected by the model and that the model was designed to handle. On the other hand, a random direction with equal magnitude would be more disruptive to the residual stream because it would move it off of the activation manifold where the model was trained to operate. This interpretation aligns with the observation that SAE features represent directional structure within the residual-space of the stream of residuals (Cunningham et al., 2023; Park et al., 2024). Furthermore, even though this is weak evidence, it also represents some form of non-trivial evidence supporting the notion that the SAE features utilized in Chapter 3 do not simply represent arbitrary representations but have correspondence to something the model does use.

Two significant limitations of the current check. Firstly, KL divergence is a “blunt” metric; i.e., it will tell you if your target distribution has changed from your baseline distribution, but it doesn’t give you information about what caused the change, nor does it tell you if the change happened along lines that are consistent with the intended semantic interpretation of the features being used, versus some other unconnected form of interference. In order to perform behaviourally-grounded causal validation of the wrapper hypothesis, we needed to implement a task-specific read-out (i.e., an objective measure of how performance changes when one uses the steering mechanism on JailbreakBench to steer the safety-feature, or on a follow-the-instruction task to steer the instruction-feature). However, such a read-out was not implemented. Secondly, this check only provides structural evidence of whether there exists a structural difference between the true direction and random direction, not semantic evidence regarding what these differences signify. Thus, our results represent a very weak structural observation and we do not interpret them as providing support either for or against the wrapper

hypothesis. The full activation-steering experiment that would provide a causal complement to Chapter 3 is discussed in Subsection 5.4 as the most important future work.

4.4 Summary of Checks Performed

Chapters 2 and 3 report four independent robustness mechanisms, each targeting a different possible failure mode. They are collected here for convenience.

1. **A 25-point threshold sensitivity grid** (Subsection 3.1). This confirms that the qualitative characteristics of the shape (preserved minority, changed dominant, exclusive classes large but not behaviorally central), were found to be stable throughout the entire grid. The primary operating point (0.7,0.3) was selected based upon its ability to represent the overall group and was not selected because it provided some unique insight into how the classifications varied.
2. **An inter-annotator reliability check of labelling vocabulary** (Sections 3.2–3.3, Subsections ?? and ??). Two LLMs (with identical prompts and schemas) generated labelling vocabularies. Agreement among raters resulted in a 60% agreement rate. Additionally, the areas where disagreements occurred were concentrated at specific boundary points. As a result of these findings, all distributional claims made in Sections 3.2–3.3 utilized category counts rather than individual label data.
3. **The targeted analysis.** Three probe datasets were used to test two behavioral contrasts—safety (JailbreakBench) and instruction-following, which compared Alpaca (Taori et al., 2023) against TruthfulQA as a factual baseline. Welch’s t -tests were used to compare differences between groups using $p < 0.01$ with a minimum activation filter applied. Because both contrasts produced similar qualitative readings, they provide evidence of cross-contrast robustness.
4. **The cross-SAE direction check** (Subsubsection 4.3). This check determined that there exists a structurally significant difference between real directions and randomly generated directions. Therefore, SAE features are identified as non-arbitrary axes of variation.

Each of the descriptive conclusions of Chapter 3 are supported through four types of robustness checks. These include: threshold stability; inter-annotator reliability; cross-contrast convergence; and real-versus-random structural distinctiveness. Each type of robustness check addresses a different potential criticism. All robustness checks have sufficient detail so as to allow reproduction. While none substitute for a behaviorally grounded causal experiment, the absence of such an experiment is the most important limitation of this thesis, named as the primary future work in Section 5.4.

5 Conclusion

Evidence from Chapter 3 supports the following descriptive and correlational conclusions. In layer 13 of Gemma 3 1B, the behaviorally relevant instructional and safety features are mainly composed of reorganized pre-existing features instead of newly added ones. The evidence does not provide sufficient support for the stronger interpretation (i.e., that there exist no new features at all), since the IT-exclusive class is not empty, and a substantial proportion of the active IT features do not correspond to a strong PT-feature. However, the evidence does provide support for the weaker interpretation: the features responsible for the behavioral signature of instruction tuning—responding to instructions and to harmful content—reside within the modified class, which, by definition, contains elements that have a corresponding element in the base model. Therefore, the evidence supports the existence of the IT-exclusive class, but shows that this is not the site of behavioral activity.

The above conclusion is compatible with the wrapper hypothesis of Jain, Kirk, et al. (2024), originally presented in synthetic environments using pruning and linear probing. This study represents an extension of the wrapper hypothesis to a real-world instruction-tuned language model, at the feature level, based on SAEs as a descriptive vocabulary and on targeted differential analysis using three probe datasets instantiating two behavioral contrasts as the behavioral basis. The extension is not causal validation (see Chapter 4), but it constitutes a meaningful descriptive result connecting the Jain et al. proposal to a production-class model. The two results that carry the burden of supporting this conclusion are:

- the cross-model feature index overlap in Section 3.3 regarding the instruction-following contrast, where the top-ranked differential features in the IT model coincide with the top-ranked features in the PT model with large effect sizes in both models; and
- the distributional pattern of Sections 3.1–3.2, where instruction-type features are concentrated in the modified class instead of either the preserved or the IT-exclusive classes.

Finally, this conclusion is also compatible with the broader body of research concerning mechanistic fine-tuning since Jain et al.: Prakash et al. (2024) show that fine-tuning enhances

existing mechanisms for entity tracking rather than introducing new ones, and the sparse-model-diffing literature has reached agreement on similar results in different model families.

5.1 Return to the Motivation and the Research Question

The thesis started from an argument on if instruction tuning reorganises existing knowledge or creates genuinely new structure. The answer supported here is the weaker reorganisation claim: instruction tuning does not appear to create most of the features carrying the behavioural signatures of instruction-following and safety, even though the IT-exclusive class is not empty.

This supports one particular conclusion regarding the superficial alignment literature. Zhou et al. (2023)’s LIMA observation states that a relatively small amount of well-performing supervised training data can produce significant amounts of instructional behavior. This suggests that the base model contains at least some portion of the required substrate for this behavior. In addition, the feature-level analysis presented above aligns with this interpretation in ways previous research did not. Specifically, the selectivity of certain SAE feature indices in the base model for instruction-shaped prompts is clearly observable, and the IT model amplifies these selections. As such, features 2006 and 446, and the other features in the cross-model overlap set defined in Section 3.3, represent individualized representational objects that superficial alignment theory claims should exist, were demonstrated to exist in the current study, and can be identified individually.

In a similar manner, the safety comparison demonstrates a parallel, yet weaker, version of this story. As stated previously, the top-ranked features associated with harmful prompts are common across all checkpoint models. Also, none of the top twenty safety features of the IT model are unique to the IT model.

5.2 Implications for Alignment Practice

If the generalizable illustration of Section 5 holds beyond the confines of this case study (a notion that this thesis supports but cannot verify), there are many consequences for how alignment will be practiced. These are discussed below as “hypotheses”—not as “claims supported by the data” collected within this work.

Base-model auditing becomes a meaningful safety intervention. If instruction tuning is essentially just reorganizing the original characteristics of the model, then the original characteristic set of the base model provides a ceiling on what the instruction-tuned model can represent. A feature not present in the base model cannot be created by simply re-weighting; and similarly, any feature that is present in the base model (regardless of its dormancy or latency) could potentially be amplified when the model is fine-tuned. Thus, if an alignment group wishes to understand which behaviors a particular instruction-tuned model may produce, they have a rational basis to audit the base model’s SAE feature space to determine which latent capabilities fine-tuning may expose. This represents a more narrow version of two concerns identified in prior literature (Hubinger et al., 2024; Zou et al., 2023): one being that behaviors not typically produced by the IT model may still be recovered via adversarial prompts, and another representing a “sleeping agent” concern where safety training may not eliminate, but instead hide, dangerous behavior. Here, the feature-level data directly supports both versions at a very narrow scale: since all the capabilities fine-tuning works upon are in the base model, any type of adversarial prompting intended to bypass fine-tuning and thus elicit behavior from the base model itself would likely find behavior in the base model rather than some feature unique to instruction tuning.

Safety fine-tuning is a reweighting, not a replacement. The findings of Section 3.3, that most IT safety features were taken from the base model population, suggest that safety behavior is represented by modulating pre-existing features, not by adding some new structure specifically for safety in the process of fine-tuning. This conclusion aligns with observations made elsewhere (Jain, Lubana, et al., 2024) that safety fine-tuning can be undone through further fine-tuning on seemingly unrelated tasks. Since we know safety is a re-weighting and not a structural addition, any additional training that alters those weights (whether or not done through adversarial processes) can plausibly degrade it. While the feature-level evidence presented does not provide proof for this mechanism, it is the type of evidence such a mechanism would predict. The results of Section 4.3 offer somewhat weaker support for this idea, suggesting that SAE directions used across checkpoints are not treated entirely as random perturbations, although these results are sufficiently ambiguous to provide little support for any

practically useful conclusions.

5.3 The Pipeline as a Reusable Methodological Contribution

What stands out as the novel methodology is that the choice was made to use dual-LLM annotator disagreement as data (signal) instead of simply as error. Since the two annotators are concentratedly disagreeing on the `language_syntax` ↔ `instruction_style` border, the labelling process is doing more than assigning names to characteristics; it is giving us an empirical measurement of when our taxonomic system becomes ambiguous. This will be valuable for making use of this data to informally identify the boundaries of our taxonomical interpretation system using multiple annotators, and is therefore the portion of the pipeline we would like to carry with us to future studies comparing matched SAEs.

5.4 Future Work

Six areas of potential direction for future research can be inferred from the findings and limitations of the paper, in descending order of their relevance.

Behaviorally grounded causal validation. The major disadvantage (Chapter 4) that has been found so far is the lack of an experimental design that tests whether the candidate features will indeed lead to the behaviors they have been selected for. A complete edition would steer each feature individually (for example, cross-modal instruction-style features 2006 and 446, or safety-related features 262 and 2184) and record specific task metrics: refusal rates on JailbreakBench when safety-related features are steered, or accuracy on Alpaca when instruction-style features are steered. The KL-based metric described in Section 4.3, however, is insufficient, since it records how much the distributions have changed but does not assess how those changes relate to the specific tasks being performed. We therefore suggest prioritizing this experiment over any other area of future work.

Multilayer replication. In Chapter 3, results were reported only at Layer 13. If the entire pipeline is rerun at every layer with identical sets of SAEs, one may obtain a view of how instruction tuning affects each layer. Such a layer profile may help determine whether instruction tuning affects the residual stream uniformly throughout the layers, concentrates at middle

layers (i.e., layers with the highest level of semantic features), or at output layers (i.e., layers where behavioral decisions are taken).

Cross-family replication. The findings reported in this paper are based on a single case study. By running the same experiments on other model families once equivalent SAEs are available, researchers may verify whether the consistent view provided by the wrapper hypothesis holds outside of DeepMind’s instruction tuning of Gemma 3 1B. Several plausible alternatives exist, including Llama 3, Mistral, and Qwen; the limiting factor is currently access to matched SAEs, not pipeline complexity.

Safety prompts near boundaries. If the same targeted differential analysis is run using prompts around the refusal boundary (i.e., prompts where minor wording changes result in compliance versus non-compliance), researchers may verify whether explicit safety features appear more prominently in the IT top lists than was observed in Subsection 3.3, using a stronger behavioral probe than a fixed harmful set.

Cross-checkpoint crosscoders. Instead of using two independent SAEs, a single cross-coder trained simultaneously on both PT and IT residual streams—similar to those used in the sparse-model-diffing literature—would eliminate the matched-SAE comparison problem by definition. Crosscoders were not employed in this research because the Gemma Scope 2 (McDougall et al., 2025) crosscoders released are cross-layer rather than cross-checkpoint, and no public PT $\leftrightarrow$ IT crosscoder existed. A future release would enable researchers to recompute the classification using a single common vocabulary derived from a shared dictionary, and comparing separate-SAE and crosscoder classifications would also be of methodological interest.

Threshold sensitivity for active-feature fraction. One minor unfinished piece: a formal sensitivity grid was not generated across active-feature thresholds. The classification proportions were informally checked and do not appear strongly sensitive within the 0.05%–0.5% range, but extending the grid in Subsection 2.4 to three dimensions (including this threshold) would close this loose end.

5.5 Closing Reflection

This dissertation was intended to test an argumentative issue regarding instruction tuning based on a feature-based pipeline, and it gives an illustrative description which matches the "wrapper" reading: at layer thirteen of Gemma 3 1B, the majority of the features most closely related to the instruction-following and safety prompt behaviours have undergone reorganisation of existing features and not primarily new ones. However, the answer given by the dissertation does not provide causal results. Instead, it is a concrete, reproducible piece of information relative to the feature-based side of the current debate concerning instruction-tuning that has taken place largely at the behavioral and circuit levels and a pipeline that can be run again when matched SAEs will be available for other model-pairs. The original formulation of the wrapper hypothesis was developed for synthetic tasks and tested through pruning and probing; however, this dissertation extended its application to a production-class model that had been tuned through instructions and testing of the hypothesis with dictionary learning. The convergence of independent methods on compatible readings is the most significant contribution of this thesis. Therefore, the next logical steps (i.e., validating causality, replicating across multiple layers, and replicating across different families) represent a continued development along the same lines as those upon which this dissertation builds.

References

- Anthropic. (2025, January 1). *Claude haiku 4.5*. <https://www.anthropic.com/claude/haiku>
- Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., Joseph, N., Kadavath, S., Kernion, J., Conerly, T., El-Showk, S., Elhage, N., Hatfield-Dodds, Z., Hernandez, D., Hume, T., . . . Kaplan, J. (2022, April 12). Training a helpful and harmless assistant with reinforcement learning from human feedback. <https://doi.org/10.48550/arXiv.2204.05862>
- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, D., Tran-Johnson, E., Perez, E., . . . Kaplan, J. (2022, December 15). Constitutional AI: Harmlessness from AI feedback. <https://doi.org/10.48550/arXiv.2212.08073>
- Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., Lasenby, R., Wu, Y., Kravec, S., Schiefer, N., Maxwell, T., Joseph, N., Hatfield-Dodds, Z., Tamkin, A., Nguyen, K., . . . Olah, C. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. <https://transformer-circuits.pub/2023/monosemantic-features/>
- Chao, P., DeBenedetti, E., Robey, A., Andriushchenko, M., Croce, F., Schwag, V., Dobriban, E., Flammarion, N., Pappas, G. J., Tramer, F., Hassani, H., & Wong, E. (2024, March 28). Jailbreakbench: An open robustness benchmark for jailbreaking large language models. <https://doi.org/10.48550/arXiv.2404.01318>
- Christiano, P., Leike, J., Brown, T. B., Martic, M., Legg, S., & Amodei, D. (2023, February 17). Deep reinforcement learning from human preferences. <https://doi.org/10.48550/arXiv.1706.03741>
- Cunningham, H., Ewart, A., Riggs, L., Huben, R., & Sharkey, L. (2023, October 4). Sparse autoencoders find highly interpretable features in language models. <https://doi.org/10.48550/arXiv.2309.08600>

- Du, H., Li, W., Cai, M., Saraipour, K., Zhang, Z., Lakkaraju, H., Sun, Y., & Zhang, S. (2025, August 11). How post-training reshapes LLMs: A mechanistic view on knowledge, truthfulness, refusal, and confidence [version: 2]. <https://doi.org/10.48550/arXiv.2504.02904>
- Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., Grosse, R., McCandlish, S., Kaplan, J., Amodei, D., Wattenberg, M., & Olah, C. (2022, September 21). Toy models of superposition. <https://doi.org/10.48550/arXiv.2209.10652>
- Gao, L., Tour, T. D. I., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., & Wu, J. (2024, June 6). Scaling and evaluating sparse autoencoders. <https://doi.org/10.48550/arXiv.2406.04093>
- Geva, M., Schuster, R., Berant, J., & Levy, O. (2021). Transformer feed-forward layers are key-value memories. In M.-F. Moens, X. Huang, L. Specia, & S. W.-t. Yih (Eds.), *Proceedings of the 2021 conference on empirical methods in natural language processing* (pp. 5484–5495). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.emnlp-main.446>
- Google. (2025a). *Google/gemma-3-1b-it: Hugging face model card*. Retrieved April 10, 2026, from <https://huggingface.co/google/gemma-3-1b-it>
- Google. (2025b). *Google/gemma-3-1b-pt: Hugging face model card*. Retrieved April 10, 2026, from <https://huggingface.co/google/gemma-3-1b-pt>
- Hubinger, E., Denison, C., Mu, J., Lambert, M., Tong, M., MacDiarmid, M., Lanham, T., Ziegler, D. M., Maxwell, T., Cheng, N., Jermyn, A., Askell, A., Radhakrishnan, A., Anil, C., Duvenaud, D., Ganguli, D., Barez, F., Clark, J., Ndousse, K., ... Perez, E. (2024, January 17). Sleeper agents: Training deceptive LLMs that persist through safety training. <https://doi.org/10.48550/arXiv.2401.05566>
- Jain, S., Kirk, R., Lubana, E. S., Dick, R. P., Tanaka, H., Grefenstette, E., Rocktäschel, T., & Krueger, D. S. (2024, August 21). Mechanistically analyzing the effects of fine-tuning on procedurally defined tasks. <https://doi.org/10.48550/arXiv.2311.12786>

- Jain, S., Lubana, E. S., Oksuz, K., Joy, T., Torr, P. H. S., Sanyal, A., & Dokania, P. K. (2024). What makes and breaks safety fine-tuning? a mechanistic study. <https://arxiv.org/abs/2407.10264>
- Karvonen, A., Rager, C., Lin, J., Tigges, C., Bloom, J., Chanin, D., Lau, Y.-T., Farrell, E., McDougall, C., Ayonrinde, K., Till, D., Wearden, M., Conmy, A., Marks, S., & Nanda, N. (2025, June 4). SAEBench: A comprehensive benchmark for sparse autoencoders in language model interpretability. <https://doi.org/10.48550/arXiv.2503.09532>
- Kornblith, S., Norouzi, M., Lee, H., & Hinton, G. (2019, July 19). Similarity of neural network representations revisited. <https://doi.org/10.48550/arXiv.1905.00414>
- Lieberum, T., Rajamanoharan, S., Conmy, A., Smith, L., Sonnerat, N., Varma, V., Kramár, J., Dragan, A., Shah, R., & Nanda, N. (2024, August 19). Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2. <https://doi.org/10.48550/arXiv.2408.05147>
- Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA: Measuring how models mimic human falsehoods. <https://doi.org/10.48550/arXiv.2109.07958>
- McDougall, C., Conmy, A., Kramár, J., Lieberum, T., Rajamanoharan, S., & Nanda, N. (2025, September 16). *Gemma scope 2 - technical paper*. Google DeepMind. https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/gemma-scope-2-helping-the-ai-safety-community-deepen-understanding-of-complex-language-model-behavior/Gemma_Scope_2_Technical_Paper.pdf
- Meng, K., Bau, D., Andonian, A., & Belinkov, Y. (2023, January 13). Locating and editing factual associations in GPT. <https://doi.org/10.48550/arXiv.2202.05262>
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013, September 7). Efficient estimation of word representations in vector space. <https://doi.org/10.48550/arXiv.1301.3781>
- Ng, A. (2011). Sparse Autoencoder. https://web.stanford.edu/class/cs294a/sparseAutoencoder_2011new.pdf
- Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., & Carter, S. (2020). Zoom in: An introduction to circuits. *Distill*, 5(3), 10.23915/distill.00024.001. <https://doi.org/10.23915/distill.00024.001>

- Olah, C., Mordvintsev, A., & Schubert, L. (2017). Feature visualization. *Distill*, 2(11), 10.23915/distill.00007. <https://doi.org/10.23915/distill.00007>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askeel, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022, March 4). Training language models to follow instructions with human feedback. <https://doi.org/10.48550/arXiv.2203.02155>
- Park, K., Choe, Y. J., & Veitch, V. (2024, July 17). The linear representation hypothesis and the geometry of large language models. <https://doi.org/10.48550/arXiv.2311.03658>
- Penedo, G., Kydlíček, H., Lozhkov, A., Mitchell, M., Wolf, T., & von Werra, L. (2024). The FineWeb datasets: Decanting the web for the finest text data at scale. <https://doi.org/10.48550/arXiv.2406.17557>
- Peng, K., Movva, R., Kleinberg, J., Pierson, E., & Garg, N. (2025, June 30). Use sparse autoencoders to discover unknown concepts, not to act on known concepts. <https://doi.org/10.48550/arXiv.2506.23845>
- Prakash, N., Shaham, T. R., Haklay, T., Belinkov, Y., & Bau, D. (2024, February 22). Fine-tuning enhances existing mechanisms: A case study on entity tracking. <https://doi.org/10.48550/arXiv.2402.14811>
- Radford, A., Jozefowicz, R., & Sutskever, I. (2017, April 6). Learning to generate reviews and discovering sentiment. <https://doi.org/10.48550/arXiv.1704.01444>
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2024, July 29). Direct preference optimization: Your language model is secretly a reward model. <https://doi.org/10.48550/arXiv.2305.18290>
- Rajamanoharan, S., Conmy, A., Smith, L., Lieberum, T., Varma, V., Kramár, J., Shah, R., & Nanda, N. (2024, April 30). Improving dictionary learning with gated sparse autoencoders. <https://doi.org/10.48550/arXiv.2404.16014>
- Scherlis, A., Sachan, K., Jermyn, A. S., Benton, J., & Shlegeris, B. (2025, March 25). Polysemaniticity and capacity in neural networks. <https://doi.org/10.48550/arXiv.2210.01892>

- Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., & Hashimoto, T. B. (2023). Stanford alpaca [Instruction-following dataset repository]. https://github.com/tatsu-lab/stanford_alpaca
- Wang, K., Variengien, A., Conmy, A., Shlegeris, B., & Steinhardt, J. (2022, November 1). Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. <https://doi.org/10.48550/arXiv.2211.00593>
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., & Le, Q. V. (2022, February 8). Finetuned language models are zero-shot learners. <https://doi.org/10.48550/arXiv.2109.01652>
- Welch, B. L. (1947). The generalization of “student’s” problem when several different population variances are involved. *Biometrika*, 34(1-2), 28–35. <https://doi.org/10.1093/biomet/34.1-2.28>
- Xu, Y., Wang, Y., Huang, H., & Wang, H. (2025). Tracking the feature dynamics in LLM training: A mechanistic study. Retrieved 2026, from <https://arxiv.org/abs/2412.17626>
- Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., & Levy, O. (2023, May 18). LIMA: Less is more for alignment. <https://doi.org/10.48550/arXiv.2305.11206>
- Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., Christiano, P., & Irving, G. (2020, January 8). Fine-tuning language models from human preferences. <https://doi.org/10.48550/arXiv.1909.08593>
- Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A.-K., Goel, S., Li, N., Byun, M. J., Wang, Z., Mallen, A., Basart, S., Koyejo, S., Song, D., Fredrikson, M., ... Hendrycks, D. (2025, March 3). Representation engineering: A top-down approach to AI transparency. <https://doi.org/10.48550/arXiv.2310.01405>
- Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., & Fredrikson, M. (2023, December 20). Universal and transferable adversarial attacks on aligned language models. <https://doi.org/10.48550/arXiv.2307.15043>